

Store data in ViewModel

1. Before you begin
2. Starter app overview
3. Learn about App Architecture
4. Add a ViewModel
5. Move data to the ViewModel
6. The lifecycle of a ViewModel
7. Populate ViewModel
8. Dialogs
9. Implement OnClickListener for Submit button
10. Implement the Skip button
11. Verify the ViewModel preserves data
12. Update game restart logic
13. Solution code
14. Summary
15. Learn more

1. Before you begin

You have learned in the previous codelabs the lifecycle of activities and fragments and the related lifecycle issues with configuration changes. To save the app data, saving the instance state is one option, but it comes with its own limitations. In this codelab you learn about a robust way to design your app and preserve app data during configuration changes, by taking advantage of Android Jetpack libraries.

[Android Jetpack](#) libraries are a collection of libraries to make it easier for you to develop great Android apps. These libraries help you follow best practices, free you from writing boilerplate code, and simplify complex tasks, so you can focus on the code you care about, like the app logic.

[Android Architecture Components](#) are part of [Android Jetpack](#) libraries, to help you design apps with good architecture. Architecture Components provide guidance on app architecture, and it is the recommended best practice.

App architecture is a set of design rules. Much like the blueprint of a house, your architecture provides the structure for your app. A good app architecture can make your code robust, flexible, scalable and maintainable for years to come.

In this codelab, you learn how to use [ViewModel](#), one of the Architecture components to store your app data. The stored data is not lost if the framework destroys and re-creates the activities and fragments during a configuration change or other events.

Prerequisites

- How to download source code from GitHub and open it in Android Studio.
- How to create and run a basic Android app in Kotlin, using activities and fragments.
- Knowledge about Material text field and common UI widgets such as `TextView` and `Button`.
- How to use view binding in the app.
- Basics of activity and fragment lifecycle.
- How to add logging information to an app and read logs using **Logcat** in Android Studio.

What you'll learn

- Introduction to the basics of [Android app architecture](#).
- How to use the [ViewModel](#) class in your app.
- How to retain UI data through device-configuration changes using a [ViewModel](#).
- Backing properties in Kotlin.
- How to use [MaterialAlertDialog](#) from the Material Design Components library.

What you'll build

- An [Unscramble](#) game app where the user can guess the scrambled words.

What you need

- A computer with Android Studio installed.
- [Starter code](#) for the Unscramble app.

2. Starter app overview

Game overview

The Unscramble app is a single player word scrambler game. The app displays one scrambled word at a time, and the player has to guess the word using all the letters from the scrambled word. The player scores points if the word is correct, otherwise the player can try any number of times. The app also has an option to skip the current word. In the left top corner, the app displays the word count, which is the number of words played in this current game. There are 10 words per game.

9:24



Unscramble

1 of 10 words

SCORE: 0

dniwer

Unscramble the word using all the letters.

SKIP

SUBMIT



9:24



Unscramble

6 of 10 words

SCORE: 20

htisr

Unscramble the word using all the letters.

Enter your word

htidr



Try again!

SKIP

SUBMIT



4:48



Unscramble

rlmsaoocs

Unscramble the word using all the letters.

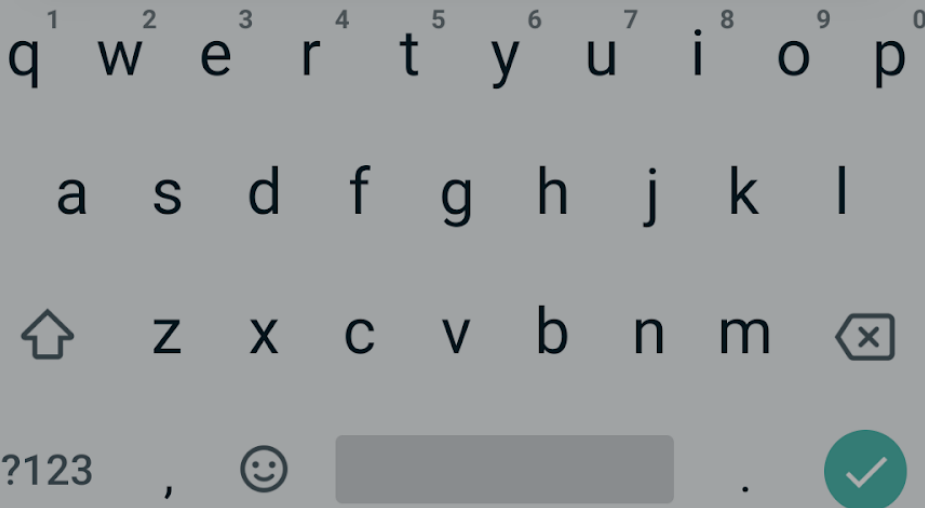
Enter your word

Congratulations!

You scored: 100

EXIT

PLAY AGAIN



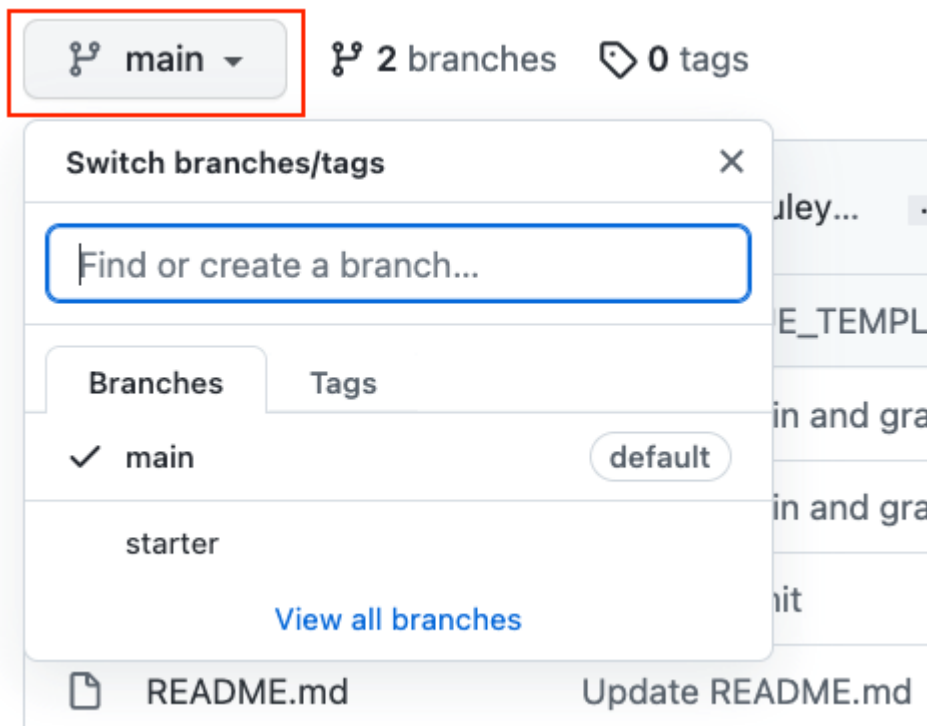
Download starter code

This codelab provides starter code for you to extend with features taught in this codelab. Starter code may contain code that is both familiar and unfamiliar to you from previous codelabs. You will learn more about unfamiliar code in later codelabs.

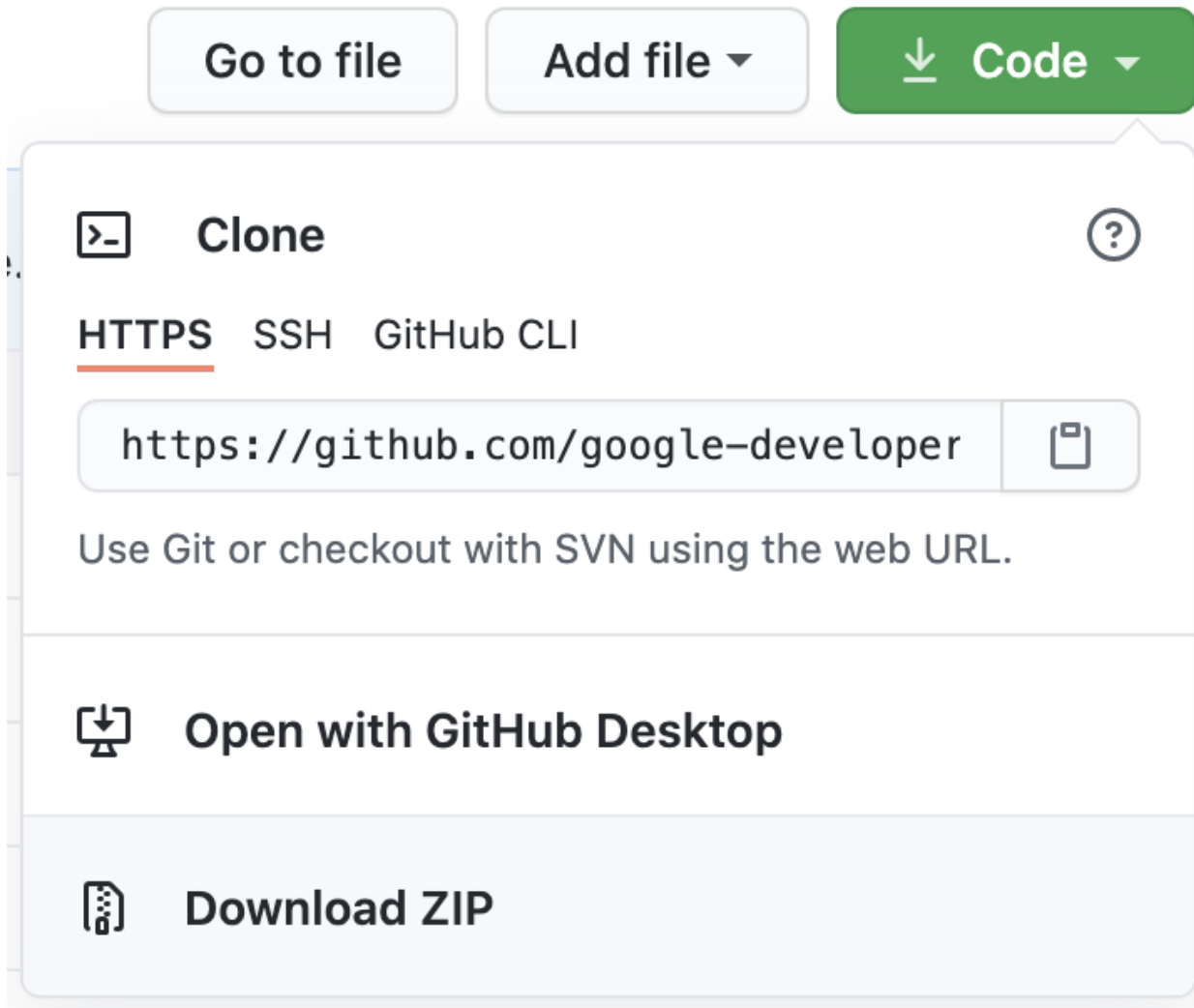
If you use the starter code from GitHub, note that the folder name is `android-basics-kotlin-unsramble-app-starter`. Select this folder when you open the project in Android Studio.

Starter Code URL: <https://github.com/google-developer-training/android-basics-kotlin-unsramble-app/tree/starter>

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



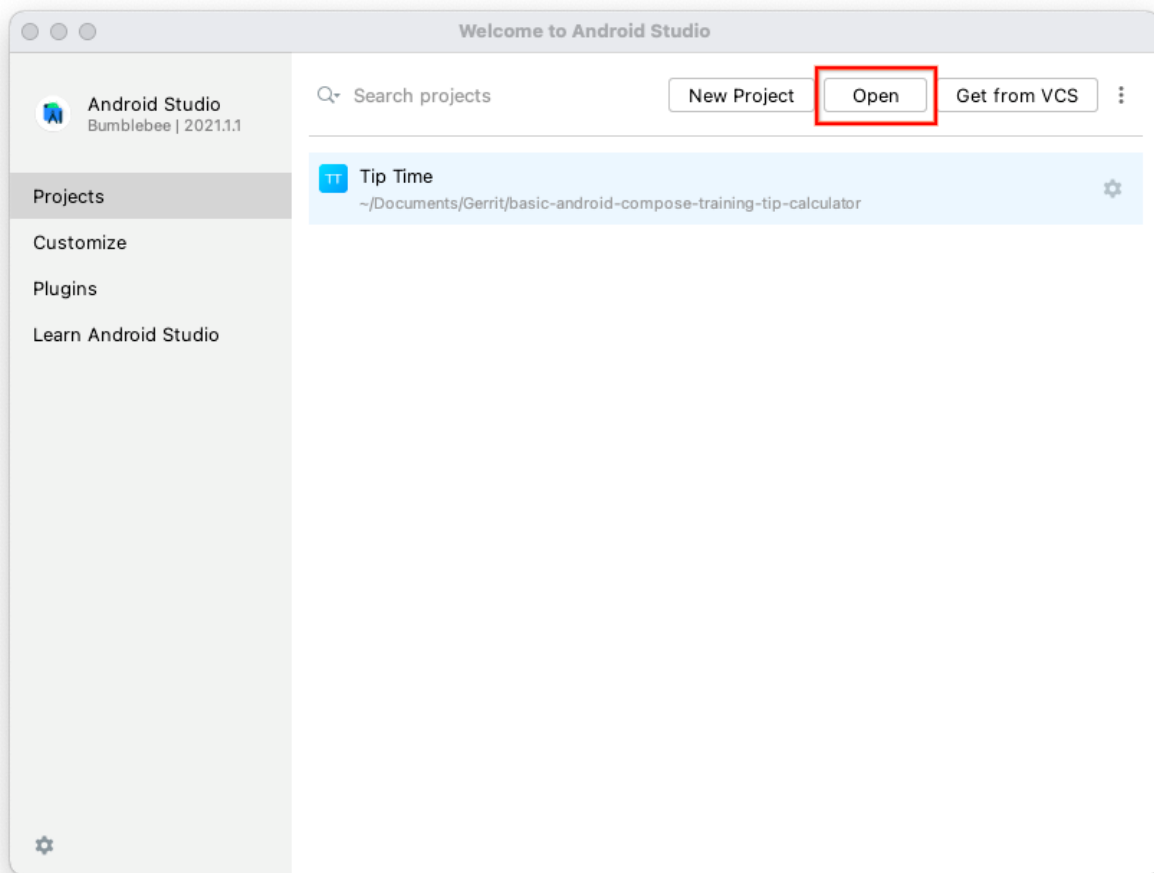
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



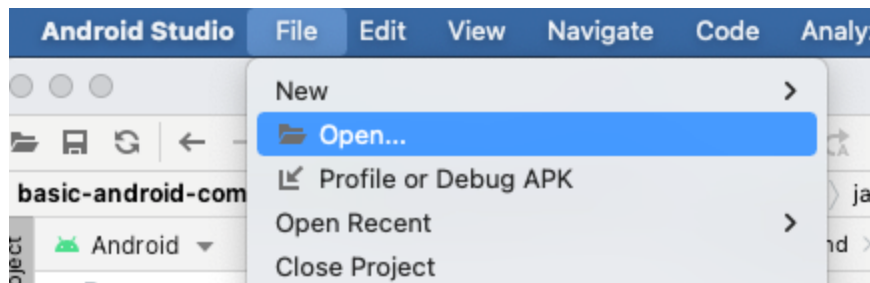
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

Starter code overview

1. Open the project with the starter code in Android Studio.
2. Run the app on an Android device, or on an emulator.
3. Play the game through a few words, tapping **Submit** and **Skip** buttons. Notice that tapping the buttons displays the next word and increases the word count.
4. Observe that the score is increased only on tapping the **Submit** button.

Problems with the starter code

As you played the game, you may have observed the following bugs:

1. On clicking the **Submit** button, the app does not check the player's word. The player always scores points.
2. There is no way to end the game. The app lets you play beyond 10 words.
3. The game screen shows a scrambled word, player's score, and word count. Change the screen orientation by rotating the device or emulator. Notice that the current word, score, and word count are lost and the game restarts from the beginning.

Main issues in the app

The starter app doesn't save and restore the app state and data during configuration changes, such as when the device orientation changes.

You could resolve this issue using the [onSaveInstanceState\(\)](#) callback. However, using the `onSaveInstanceState()` method requires you to write extra code to save the state in a bundle, and to implement logic to retrieve that state. Also, the amount of data that can be stored is minimal.

You can resolve these issues using the [Android Architecture components](#) that you learn about in this pathway.

Starter code walk through

The starter code you downloaded has the game screen layout pre-designed for you. In this pathway, you will focus on implementing the game logic. You will use architecture components to implement the recommended app architecture and resolve the above mentioned issues. Here is a brief walkthrough of some of the files to get you started.

game_fragment.xml

- Open `res/layout/game_fragment.xml` in **Design** view.
- This contains the layout of the only screen in your app that is the game screen.

- This layout contains a text field for the player's word, along with `TextViews` to display score and word count. It also has instructions and buttons (**Submit** and **Skip**) to play the game.

main_activity.xml

Defines the main activity layout with a single game fragment.

res/values folder

You are familiar with the resource files in this folder.

- `colors.xml` contains the theme colors used in the app
- `strings.xml` contains all the strings your app needs
- `themes` and `styles` folders contain the UI customization done for your app

MainActivity.kt

Contains the default template generated code to set the activity's content view as `main_activity.xml`.

ListOfWords.kt

This file contains a list of the words used in the game, as well as constants for the maximum number of words per game and the number of points the player scores for every correct word.

WARNING: It is not a recommended practice to hardcode strings in the code, strings should be in `strings.xml` for easier localization. To keep things simple, and to focus on Architecture Components, strings are hardcoded in this app.

GameFragment.kt

This is the only fragment in your app, where most of the game's action takes place:

- Variables are defined for the current scrambled word (`currentScrambledWord`), word count (`currentWordCount`), and the score (`score`).
- Binding object instance with access to the `game_fragment` views called `binding` is defined.
- `onCreateView()` function inflates the `game_fragment` layout XML using the binding object.
- `onViewCreated()` function sets up the button click listeners and updates the UI.
- `onSubmitWord()` is the click listener for the **Submit** button, this function displays the next scrambled word, clears the text field, and increases the score and word count without validating the player's word.
- `onSkipWord()` is the click listener for the **Skip** button, this function updates the UI similar to `onSubmitWord()` except the score.

- `getNextScrambledWord()` is a helper function that picks a random word from the list of words and shuffles the letters in it.
- `restartGame()` and `exitGame()` functions are used to restart and end the game respectively, you will use these functions later.
- `setErrorTextField()` clears the text field content and resets the error status.
- `updateNextWordOnScreen()` function displays the new scrambled word.

3. Learn about App Architecture

Architecture provides you with the guidelines to help you allocate responsibilities in your app, between the classes. A well-designed app architecture helps you scale your app and extend it with additional features in the future. It also makes team collaboration easier.

The most common [architectural principles](#) are: separation of concerns and driving UI from a model.

Separation of concerns

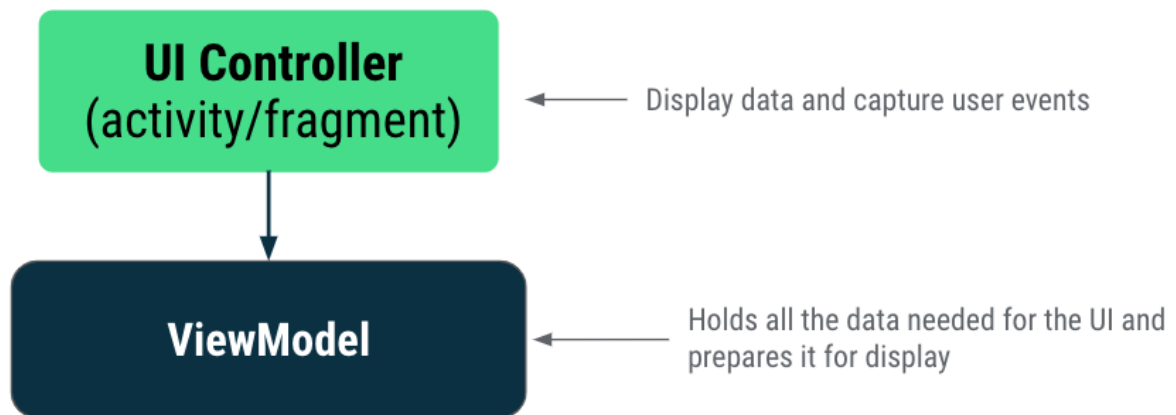
The separation of concerns design principle states that the app should be divided into classes, each with separate responsibilities.

Drive UI from a model

Another important principle is that you should drive your UI from a model, preferably a persistent model. Models are components that are responsible for handling the data for an app. They're independent from the `Views` and app components in your app, so they're unaffected by the app's lifecycle and the associated concerns.

The main classes or components in Android Architecture are UI Controller (activity/fragment), `ViewModel`, `LiveData` and `Room`. These components take care of some of the complexity of the lifecycle and help you avoid lifecycle related issues. You learn about `LiveData` and `Room` in later codelabs.

This diagram shows a basic portion of the architecture:



UI controller (Activity / Fragment)

Activities and fragments are UI controllers. UI controllers control the UI by drawing views on the screen, capturing user events, and anything else related to the UI that the user interacts with. Data in the app or any decision-making logic about that data should not be in the UI controller classes.

The Android system can destroy UI controllers at any time based on certain user interactions or because of system conditions like low memory. Because these events aren't under your control, you shouldn't store any app data or state in UI controllers. Instead, the decision-making logic about the data should be added in your `ViewModel`.

For example, in your **Unscramble** app, the scrambled word, score, and word count are displayed in a fragment (UI controller). The decision-making code such as figuring out the next scrambled word, and calculations of score and word count should be in your `ViewModel`.

ViewModel

The [ViewModel](#) is a model of the app data that is displayed in the views. Models are components that are responsible for handling the data for an app. They allow your app to follow the architecture principle, driving the UI from the model.

The [ViewModel](#) stores the app related data that isn't destroyed when activity or fragment is destroyed and recreated by the Android framework. `ViewModel` objects are automatically retained (they are not destroyed like the activity or a fragment instance) during configuration changes so that data they hold is immediately available to the next activity or fragment instance.

To implement `ViewModel` in your app, extend the `ViewModel` class, which is from the architecture components library, and store app data within that class.

To summarize:

Fragment / activity (UI controller) responsibilities

Activities and fragments are responsible for drawing views and data to the screen and responding to the user events.

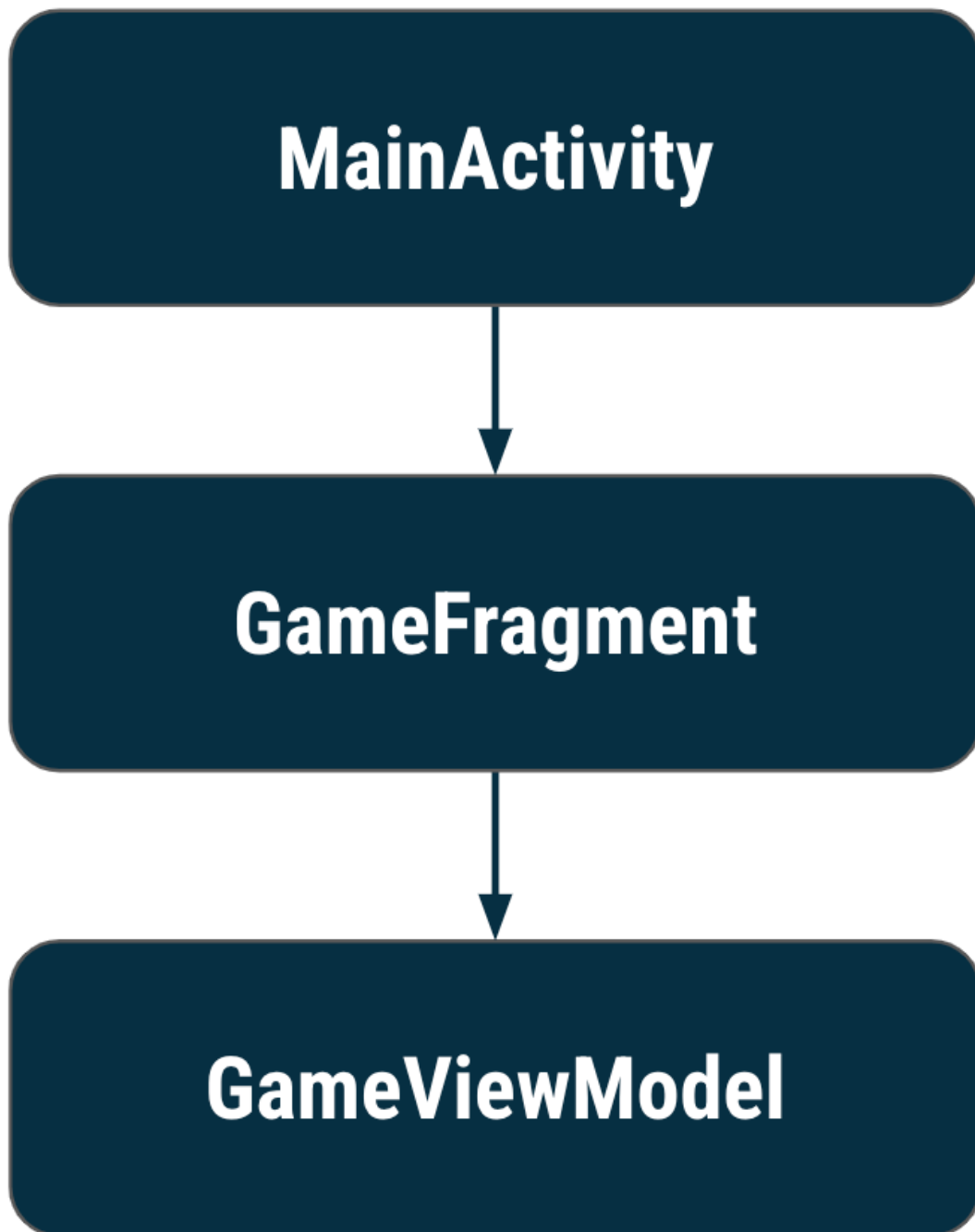
ViewModel responsibilities

ViewModel is responsible for holding and processing all the data needed for the UI. It should never access your view hierarchy (like view binding object) or hold a reference to the activity or the fragment.

4. Add a ViewModel

In this task, you add a `ViewModel` to your app to store your app data (scrambled word, word count, and score).

Your app will be architected in the following way. `MainActivity` contains a `GameFragment`, and the `GameFragment` will access information about the game from the `GameViewModel`.

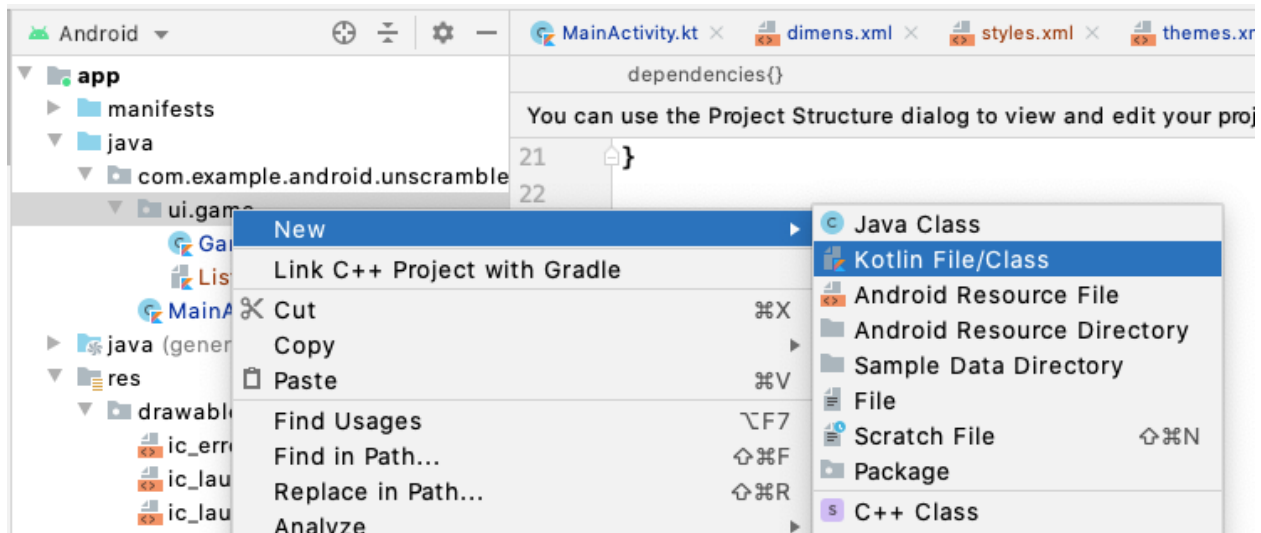


1. In the **Android** window of your Android Studio under the **Gradle Scripts** folder, open the file `build.gradle (Module:Unscramble.app)`.
2. To use the `ViewModel` in your app, verify that you have the `ViewModel` library dependency inside the `dependencies` block. This step is already done for you. Depending on the latest version of the library, the library version number in the generated code might be different.

```
// ViewModel
implementation 'androidx.lifecycle:lifecycle-viewmodel-ktx:2.3.1'
```

It is recommended to always use the [latest version](#) of the library in spite of the version mentioned in the codelab.

3. Create a new Kotlin class file called `GameViewModel`. In the **Android** window, right click on the **ui.game** folder. Select **New > Kotlin File/Class**.



4. Give it the name `GameViewModel`, and select **Class** from the list.
5. Change `GameViewModel` to be subclassed from `ViewModel`. `ViewModel` is an abstract class, so you need to extend it to use it in your app. See the `GameViewModel` class definition below.

```
class GameViewModel : ViewModel() {
}
```

Attach the ViewModel to the Fragment

To associate a `ViewModel` to a UI controller (activity / fragment), create a reference (object) to the `ViewModel` inside the UI controller.

In this step, you create an object instance of the `GameViewModel` inside the corresponding UI controller, which is `GameFragment`.

1. At the top of the `GameFragment` class, add a property of type `GameViewModel`.
2. Initialize the `GameViewModel` using the `by viewModels()` Kotlin property delegate. You will learn more about it in the next section.

```
private val viewModel: GameViewModel by viewModels()
```

3. If prompted by Android Studio, import `androidx.fragment.app.viewModels`.

Kotlin property delegate

In Kotlin, each mutable (`var`) property has default getter and setter functions automatically generated for it. The setter and getter functions are called when you assign a value or read the value of the property.

For a read-only property (`val`), it differs slightly from a mutable property. Only the getter function is generated by default. This getter function is called when you read the value of a read-only property.

Property delegation in Kotlin helps you to handoff the getter-setter responsibility to a different class.

This class (called *delegate class*) provides getter and setter functions of the property and handles its changes.

A delegate property is defined using the `by` clause and a delegate class instance:

```
// Syntax for property delegation
var <property-name> : <property-type> by <delegate-class>()
```

In your app, if you initialize the view model using default `GameViewModel` constructor, like below:

```
private val viewModel = GameViewModel()
```

Then the app will lose the state of the `viewModel` reference when the device goes through a configuration change. For example, if you rotate the device, then the activity is destroyed and created again, and you'll have a new view model instance with the initial state again.

Instead, use the property delegate approach and delegate the responsibility of the `viewModel` object to a separate class called `viewModels`. That means when you access the `viewModel` object, it is handled internally by the delegate class, `viewModels`. The delegate class creates the `viewModel` object for you on the first access, and retains its value through configuration changes and returns the value when requested.

5. Move data to the ViewModel

Separating your app's UI data from the UI controller (your `Activity / Fragment` classes) lets you better follow the single responsibility principle we discussed above. Your activities and fragments are responsible for drawing views and data to the screen, while your `ViewModel` is responsible for holding and processing all the data needed for the UI.

In this task, you move the data variables from `GameFragment` to `GameViewModel` class.

1. Move the data variables `score`, `currentWordCount`, `currentScrambledWord` to `GameViewModel` class.

```
class GameViewModel : ViewModel() {  
  
    private var score = 0  
    private var currentWordCount = 0  
    private var currentScrambledWord = "test"  
    ...  
}
```

2. Notice the errors about unresolved references. This is because properties are private to the `ViewModel` and are not accessible by your UI controller. You'll fix these errors next.

To resolve this issue, you can't make the visibility modifiers of the properties `public`—the data should not be editable by other classes. This is risky because an outside class could change the data in unexpected ways that don't follow the game rules specified in the view model. For example, an outside class could change the `score` to a negative value.

Inside the `ViewModel`, the data should be editable, so they should be `private` and `var`. From outside the `ViewModel`, data should be readable, but not editable, so the data should be exposed as `public` and `val`. To achieve this behavior, Kotlin has a feature called a [backing property](#).

Backing property

A backing property allows you to return something from a getter other than the exact object.

You have already learned that for every property, the Kotlin framework generates getters and setters.

For getter and setter methods, you could override one or both of these methods and provide your own custom behavior. To implement a backing property, you will override the getter method to return a read-only version of your data. Example of backing property:

```
// Declare private mutable variable that can only be modified  
// within the class it is declared.  
private var _count = 0  
  
// Declare another public immutable field and override its getter method.  
// Return the private property's value in the getter method.  
// When count is accessed, the get() function is called and  
// the value of _count is returned.  
val count: Int  
    get() = _count
```

Consider an example, in your app you want the app data to be private to the `ViewModel`:

Inside the `ViewModel` class:

- The property `_count` is `private` and mutable. Hence, it is only accessible and editable within the `ViewModel` class. The convention is to prefix the `private` property with an underscore.

Outside the `ViewModel` class:

- The default visibility modifier in Kotlin is `public`, so `count` is public and accessible from other classes like UI controllers. Since only the `get()` method is being overridden, this property is immutable and read-only. When an outside class accesses this property, it returns the value of `_count` and its value can't be modified. This protects the app data inside the `ViewModel` from unwanted and unsafe changes by external classes, but it allows external callers to safely access its value.

Add backing property to `currentScrambledWord`

1. In `GameViewModel` change the `currentScrambledWord` declaration to add a backing property. Now `_currentScrambledWord` is accessible and editable only within the `GameViewModel`. The UI controller, `GameFragment` can read its value using the read-only property, `currentScrambledWord`.

```
private var _currentScrambledWord = "test"
val currentScrambledWord: String
    get() = _currentScrambledWord
```

2. In `GameFragment`, update the method `updateNextWordOnScreen()` to use the read-only `viewModel` property, `currentScrambledWord`.

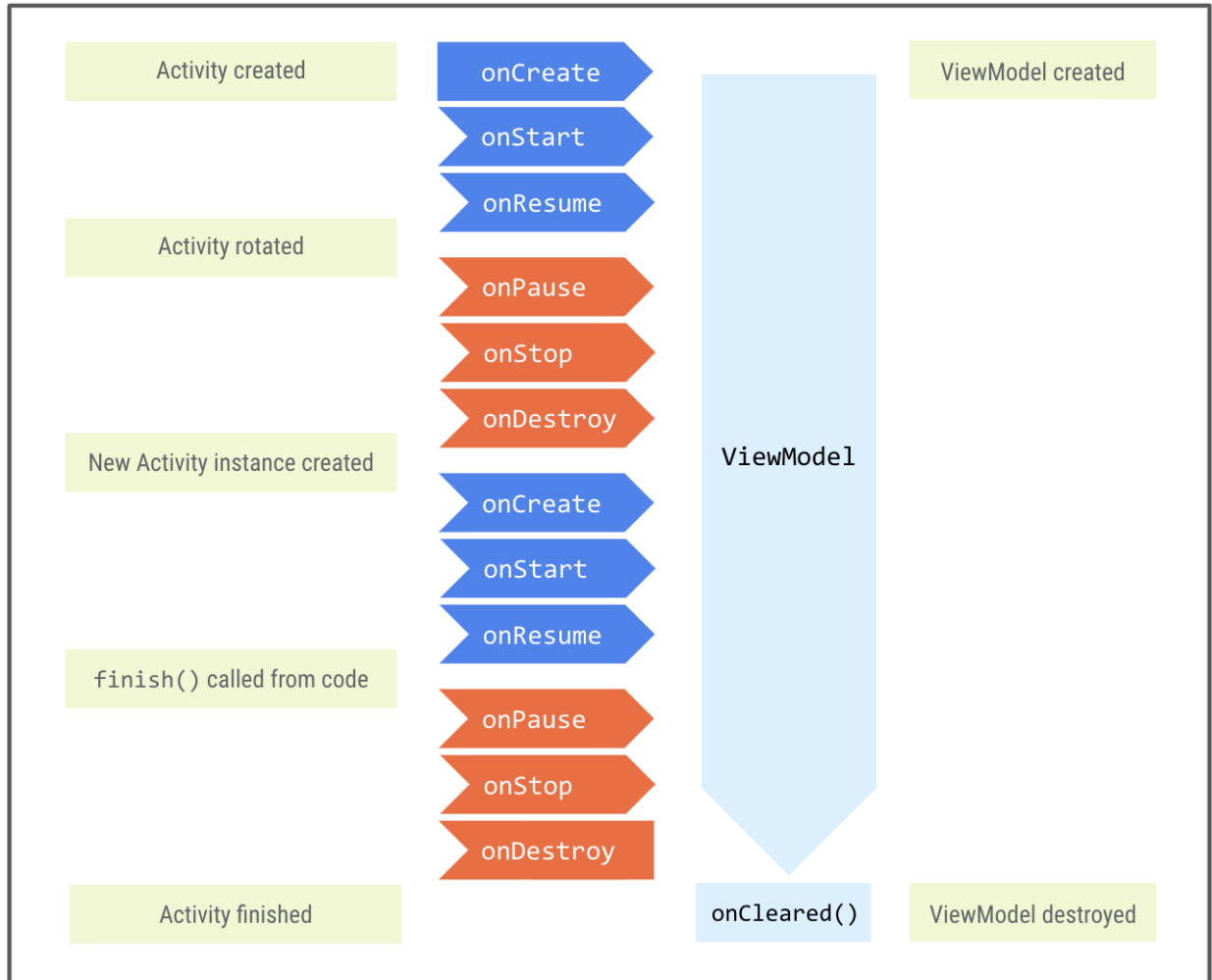
```
private fun updateNextWordOnScreen() {
    binding.textViewUnscrambledWord.text = viewModel.currentScrambledWord
}
```

3. In `GameFragment`, delete the code inside the methods `onSubmitWord()` and `onSkipWord()`. You will implement these methods later. You should be able to compile the code now without errors.

Warning: Never expose mutable data fields from your `ViewModel`—make sure this data can't be modified from another class. Mutable data inside the `ViewModel` should always be `private`.

6. The lifecycle of a `ViewModel`

The framework keeps the [ViewModel](#) alive as long as the scope of the activity or fragment is alive. A `ViewModel` is not destroyed if its owner is destroyed for a configuration change, such as screen rotation. The new instance of the owner reconnects to the existing `ViewModel` instance, as illustrated by the following diagram:



Understand ViewModel lifecycle

Add logging in the `GameViewModel` and `GameFragment` to help you better understand the lifecycle of the `ViewModel`.

1. In `GameViewModel.kt` add an `init` block with a log statement.

```
class GameViewModel : ViewModel() {  
    init {  
        Log.d("GameFragment", "GameViewModel created!")  
    }  
  
    ...  
}
```

Kotlin provides the initializer block (also known as the `init` block) as a place for initial setup code needed during the initialization of an object instance. Initializer blocks are prefixed with the

`init` keyword followed by the curly braces `{}`. This block of code is run when the object instance is first created and initialized.

2. In the `GameViewModel` class, override the `onCleared()` method. The `ViewModel` is destroyed when the associated fragment is detached, or when the activity is finished. Right before the `ViewModel` is destroyed, the `onCleared()` callback is called.
3. Add a log statement inside `onCleared()` to track the `GameViewModel` lifecycle.

```
override fun onCleared() {  
    super.onCleared()  
    Log.d("GameFragment", "GameViewModel destroyed!")  
}
```

4. In `GameFragment` inside `onCreateView()`, after you get a reference to the binding object, add a log statement to log the creation of the fragment. The `onCreateView()` callback will be triggered when the fragment is created for the first time and also every time it is re-created for any events like configuration changes.

```
override fun onCreateView(  
    inflater: LayoutInflater, container: ViewGroup?,  
    savedInstanceState: Bundle?  
): View {  
    binding = GameFragmentBinding.inflate(inflater, container, false)  
    Log.d("GameFragment", "GameFragment created/re-created!")  
    return binding.root  
}
```

5. In `GameFragment`, override the `onDetach()` callback method, which will be called when the corresponding activity and fragment are destroyed.

```
override fun onDetach() {  
    super.onDetach()  
    Log.d("GameFragment", "GameFragment destroyed!")  
}
```

6. In Android Studio, run the app, open the **Logcat** window and filter on `GameFragment`. Notice that `GameFragment` and the `GameViewModel` are created.

```
com.example.android.unscramble D/GameFragment: GameFragment created/re-  
created!  
com.example.android.unscramble D/GameFragment: GameViewModel created!
```

7. Enable the auto-rotate setting on your device or emulator and change the screen orientation a few times. The `GameFragment` is destroyed and recreated each time, but the `GameViewModel` is created only once, and it is not re-created or destroyed for each call.

```
com.example.android.unscramble D/GameFragment: GameFragment created/re-  
created!  
com.example.android.unscramble D/GameFragment: GameViewModel created!  
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
```

```
com.example.android.unscramble D/GameFragment: GameFragment created/re-created!
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-created!
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-created!
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-created!
```

8. Exit the game or navigate out of the app using the back arrow. The `GameViewModel` is destroyed, and the callback `onCleared()` is called. The `GameFragment` is destroyed.

```
com.example.android.unscramble D/GameFragment: GameViewModel destroyed!
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
```

7. Populate ViewModel

In this task, you further populate the `GameViewModel` with helper methods for getting the next word, validating the player's word to increase the score, and checking the word count to end the game.

Late initialization

Typically when you declare a variable, you provide it with an initial value upfront. However, if you're not ready to assign a value yet, you could initialize it later. To late initialize a property in Kotlin you use the keyword `lateinit`, which means late initialization. If you guarantee that you will initialize the property before using it, you can declare the property with `lateinit`. Memory is not allocated to the variable until it is initialized. If you try to access the variable before initializing it, the app will crash.

Get next word

Create the `getNextWord()` method in the `GameViewModel` class, with the following functionality:

- Get a random word from the `allWordsList` and assign it to `currentWord`.
- Create a scrambled word by scrambling the letters in the `currentWord` and assign it to the `currentScrambledWord`
- Handle the case where the scrambled word is the same as the unscrambled word.
- Make sure you don't show the same word twice during the game.

Implement the following steps in `GameViewModel` class:

1. In `GameViewModel`, add a new class variable of type `MutableList<String>` called `wordsList`, to hold a list of words you use in the game, to avoid repetitions.
2. Add another class variable called `currentWord` to hold the word the player is trying to unscramble. Use the `lateinit` keyword since you will initialize this property later.

```
private var wordsList: MutableList<String> = mutableListOf()
private lateinit var currentWord: String
```

3. Add a new private method called `getNextWord()`, above the `init` block, with no parameters that returns nothing.
4. Get a random word from the `allWordsList` and assign it to `currentWord`.

```
private fun getNextWord() {
    currentWord = allWordsList.random()
}
```

5. In `getNextWord()`, convert the `currentWord` string to an array of characters and assign it to a new `val` called `tempWord`. To scramble the word, shuffle characters in this array using the Kotlin method, [shuffle\(\)](#).

```
val tempWord = currentWord.toCharArray()
tempWord.shuffle()
```

An `Array` is similar to a `MutableList`, but it has a fixed size when it's initialized. An `Array` cannot expand or shrink its size (you need to copy an array to resize it) whereas a `MutableList` has `add()` and `remove()` functions, so that it can increase and decrease in size.

6. Sometimes the shuffled order of characters is the same as the original word. Add the following `while` loop around the call to `shuffle`, to continue the loop until the scrambled word is not the same as the original word.

```
while (String(tempWord).equals(currentWord, false)) {
    tempWord.shuffle()
}
```

7. Add an `if-else` block to check if a word has been used already. If the `wordsList` contains `currentWord`, call `getNextWord()`. If not, update the value of `_currentScrambledWord` with the newly scrambled word, increase the word count, and add the new word to the `wordsList`.

```
if (wordsList.contains(currentWord)) {
    getNextWord()
} else {
    _currentScrambledWord = String(tempWord)
    ++currentWordCount
    wordsList.add(currentWord)
}
```

8. Here is the completed `getNextWord()` method for your reference.

```

/*
 * Updates currentWord and currentScrambledWord with the next word.
 */
private fun getNextWord() {
    currentWord = allWordsList.random()
    val tempWord = currentWord.toCharArray()
    tempWord.shuffle()

    while (String(tempWord).equals(currentWord, false)) {
        tempWord.shuffle()
    }
    if (wordsList.contains(currentWord)) {
        getNextWord()
    } else {
        _currentScrambledWord = String(tempWord)
        ++currentWordCount
        wordsList.add(currentWord)
    }
}

```

Late-initialize currentScrambledWord

Now you have created the `getNextWord()` method, to get the next scrambled word. You will make a call to it when the `GameViewModel` is initialized for the first time. Use the `init` block to initialize `lateinit` properties in the class such as the current word. The result will be that the first word displayed on the screen will be a scrambled word instead of **test**.

1. Run the app. Notice the first word is always **"test"**.
2. To display a scrambled word at the start of the app, you need to call the `getNextWord()` method, which in turn updates `currentScrambledWord`. Make a call to the method `getNextWord()` inside the `init` block of the `GameViewModel`.

```

init {
    Log.d("GameFragment", "GameViewModel created!")
    getNextWord()
}

```

3. Add the `lateinit` modifier onto the `_currentScrambledWord` property. Add an explicit mention of the data type `String`, since no initial value is provided.

```

private lateinit var _currentScrambledWord: String

```

4. Run the app. Notice a new scrambled word is displayed at the app launch. Awesome!

9:24



Unscramble

1 of 10 words

SCORE: 0

dniwer

Unscramble the word using all the letters.

SKIP

SUBMIT



Add a helper method

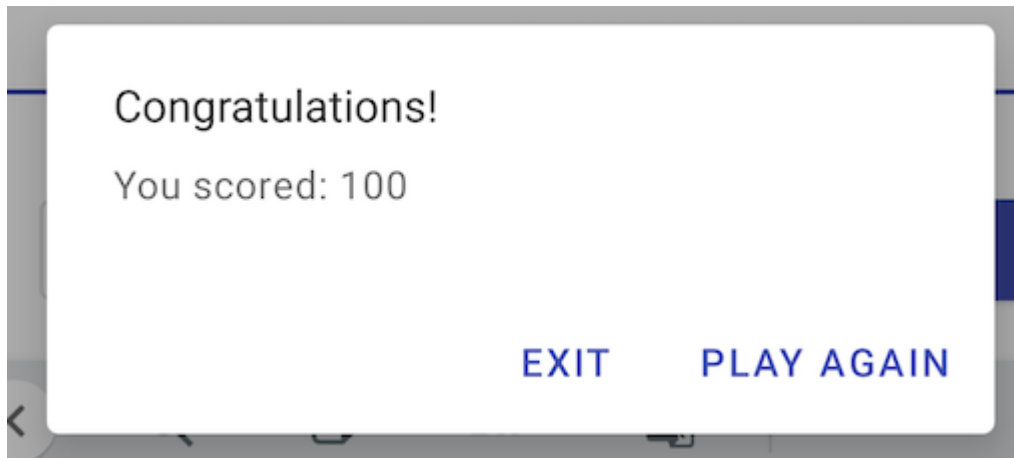
Next add a helper method to process and modify the data inside the `ViewModel`. You will use this method in later tasks.

1. In the `GameViewModel` class, add another method called `nextWord()`. Get the next word from the list and return `true` if the word count is less than the `MAX_NO_OF_WORDS`.

```
/*
 * Returns true if the current word count is less than MAX_NO_OF_WORDS.
 * Updates the next word.
 */
fun nextWord(): Boolean {
    return if (currentWordCount < MAX_NO_OF_WORDS) {
        getNextWord()
        true
    } else false
}
```

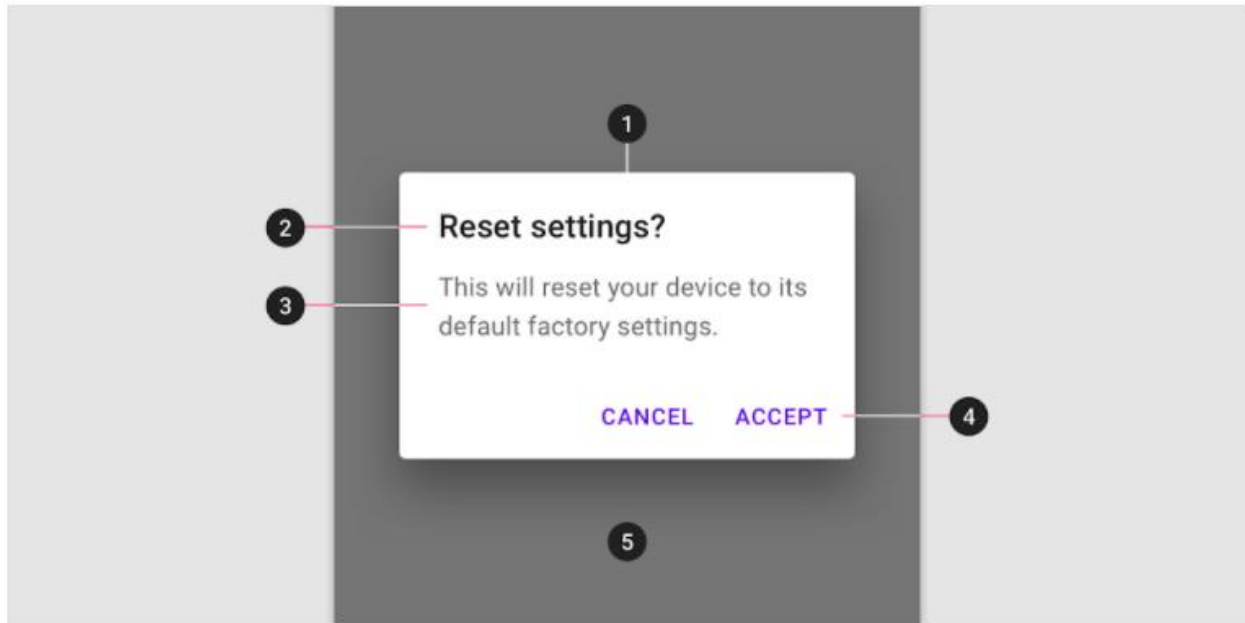
8. Dialogs

In the starter code, the game never ended, even after 10 words were played through. Modify your app so that after the user goes through 10 words, the game is over and you show a dialog with the final score. You will also give the user an option to play again or exit the game.



This is the first time you'll be adding a dialog to an app. A dialog is a small window (screen) that prompts the user to make a decision or enter additional information. Normally a dialog does not fill the entire screen, and it requires users to take an action before they can proceed. Android provides different types of Dialogs. In this codelab, you learn about Alert Dialogs.

Anatomy of alert dialog



1. Alert Dialog
2. Title (optional)
3. Message
4. Text buttons

Implement final score dialog

Use the [MaterialAlertDialog](#) from the Material Design Components library to add a dialog to your app that follows Material guidelines. Since a dialog is UI related, the `GameFragment` will be responsible for creating and showing the final score dialog.

1. First add a backing property to the `score` variable. In `GameViewModel`, change the `score` variable declaration to the following.

```
private var _score = 0
val score: Int
    get() = _score
```

2. In `GameFragment`, add a private function called `showFinalScoreDialog()`. To create a `MaterialAlertDialog`, use the `MaterialAlertDialogBuilder` class to build up parts of the dialog step-by-step. Call the [MaterialAlertDialogBuilder](#) constructor passing in the content using the fragment's [requireContext\(\)](#) method. The `requireContext()` method returns a non-null `Context`.

```
/*
 * Creates and shows an AlertDialog with the final score.
 */
private fun showFinalScoreDialog() {
```

```
MaterialAlertDialogBuilder(requireContext())
}
```

As the name suggests, [Context](#) refers to the context or the current state of an application, activity, or fragment. It contains the information regarding the activity, fragment or application. Usually it is used to get access to resources, databases, and other system services. In this step, you pass the fragment context to create the alert dialog.

If prompted by Android Studio, import
`com.google.android.material.dialog.MaterialAlertDialogBuilder.`

3. Add the code to set the title on the alert dialog, use a string resource from `strings.xml`.

```
MaterialAlertDialogBuilder(requireContext())
    .setTitle(getString(R.string.congratulations))
```

4. Set the message to show the final score, use the read-only version of the score variable (`viewModel.score`), you added earlier.

```
.setMessage(getString(R.string.you_scored, viewModel.score))
```

5. Make your alert dialog not cancelable when the back key is pressed, using [setCancelable\(\)](#) method and passing `false`.

```
.setCancelable(false)
```

6. Add two text buttons **EXIT** and **PLAY AGAIN** using the methods `setNegativeButton()` and `setPositiveButton()`. Call `exitGame()` and `restartGame()` respectively from the lambdas.

```
.setNegativeButton(getString(R.string.exit)) { _, _ ->
    exitGame()
}
.setPositiveButton(getString(R.string.play_again)) { _, _ ->
    restartGame()
}
```

This syntax may be new to you, but this is shorthand for

`setNegativeButton(getString(R.string.exit), { _, _ -> exitGame() })` where the `setNegativeButton()` method takes in two parameters: a `String` and a function, `DialogInterface.OnClickListener()` which can be expressed as a lambda. When the last argument being passed in is a function, you could place the lambda expression *outside* the parentheses. This is known as [trailing lambda syntax](#). Both ways of writing the code (with the lambda inside or outside the parentheses) is acceptable. The same applies for the `setPositiveButton` function.

7. At the end, add [show\(\)](#), which creates and then displays the alert dialog.

```
.show()
```

8. Here is the complete `showFinalScoreDialog()` method for reference.

```
/*
 * Creates and shows an AlertDialog with the final score.
 */
private fun showFinalScoreDialog() {
    MaterialAlertDialogBuilder(requireContext())
        .setTitle(getString(R.string.congratulations))
        .setMessage(getString(R.string.you_scored, viewModel.score))
        .setCancelable(false)
        .setNegativeButton(getString(R.string.exit)) { _, _ ->
            exitGame()
        }
        .setPositiveButton(getString(R.string.play_again)) { _, _ ->
            restartGame()
        }
        .show()
}
```

9. Implement OnClickListener for Submit button

In this task, you use the `ViewModel` and the alert dialog you added to implement the game logic for the **Submit** button click listener.

Display the scrambled words

1. If you haven't already done so, in `GameFragment`, delete the code inside `onSubmitWord()` which gets called when the **Submit** button is tapped.
2. Add a check on the return value of `viewModel.nextWord()` method. If `true`, another word is available, so update the scrambled word on screen using `updateNextWordOnScreen()`. Otherwise the game is over, so display the alert dialog with the final score.

```
private fun onSubmitWord() {
    if (viewModel.nextWord()) {
        updateNextWordOnScreen()
    } else {
        showFinalScoreDialog()
    }
}
```

3. Run the app! Play through some words. Remember, you have not yet implemented the **Skip** button, so you can't skip the word.
4. Notice the text field is not updated, so the player has to manually delete the previous word. The final score in the alert dialog is always zero. You will fix these bugs in the coming steps.

1:36



Unscramble

0 of 10 words

SCORE: 0

uuienq

Unscramble the word using all the letters.

Enter your word

name

SKIP

SUBMIT



1:37



Unscramble

0 of 10 words

SCORE: 0

ydti

Unscramble the word using all the letters.

Congratulations!

You scored: 0

EXIT

PLAY AGAIN

Add a helper method to validate player word

1. In `GameViewModel`, add a new private method called `increaseScore()` with no parameters and no return value. Increase the `score` variable by `SCORE_INCREASE`.

```
private fun increaseScore() {  
    _score += SCORE_INCREASE  
}
```

2. In `GameViewModel`, add a helper method called `isUserWordCorrect()` which returns a `Boolean` and takes a `String`, the player's word, as a parameter.
3. In `isUserWordCorrect()` validate the player's word and increase the score if the guess is correct. This will update the final score in your alert dialog.

```
fun isUserWordCorrect(playerWord: String): Boolean {  
    if (playerWord.equals(currentWord, true)) {  
        increaseScore()  
        return true  
    }  
    return false  
}
```

Update the text field

Show errors in text field

For Material text fields, [TextInputLayout](#) comes with a built-in functionality to display error messages. For example in the following text field, the color of the label is changed, an error icon is displayed, an error message is displayed, and so on.



To show an error in the text field, you can set the error message either dynamically in code or statically in the layout file. Example to set and reset the error in code is shown below:

```
// Set error text  
passwordLayout.error = getString(R.string.error)  
  
// Clear error text  
passwordLayout.error = null
```

In the starter code, you will find the helper method `setErrorTextField(error: Boolean)` is already defined to help you set and reset the error in the text field. Call this method with `true` or `false` as the input parameter based on whether you want an error to show up in the text field or not.

Code snippet in the starter code

```
private fun setErrorTextField(error: Boolean) {
    if (error) {
        binding.textField.isEnabled = true
        binding.textField.error = getString(R.string.try_again)
    } else {
        binding.textField.isEnabled = false
        binding.textInputEditText.text = null
    }
}
```

In this task, you implement the method `onSubmitWord()`. When a word is submitted, validate the user's guess by checking against the original word. If the word is correct, then go to the next word (or show the dialog if the game has ended). If the word is incorrect, show an error on the text field and stay on the current word.

1. In `GameFragment`, at the beginning of `onSubmitWord()`, create a `val` called `playerWord`. Store the player's word in it, by extracting it from the text field in the binding variable.

```
private fun onSubmitWord() {
    val playerWord = binding.textInputEditText.text.toString()
    ...
}
```

2. In `onSubmitWord()`, below the declaration of `playerWord`, validate the player's word. Add an `if` statement to check the player's word using the `isUserWordCorrect()` method, passing in the `playerWord`.
3. Inside the `if` block, reset the text field, call `setErrorTextField` passing in `false`.
4. Move the existing code inside the `if` block.

```
private fun onSubmitWord() {
    val playerWord = binding.textInputEditText.text.toString()

    if (viewModel.isUserWordCorrect(playerWord)) {
        setErrorTextField(false)
        if (viewModel.nextWord()) {
            updateNextWordOnScreen()
        } else {
            showFinalScoreDialog()
        }
    }
}
```

5. If the user word is incorrect, show an error message in the text field. Add an `else` block to the above `if` block, and call `setErrorTextField()` passing in `true`. Your completed `onSubmitWord()` method should look like this:

```
private fun onSubmitWord() {
    val playerWord = binding.textInputEditText.text.toString()

    if (viewModel.isUserWordCorrect(playerWord)) {
        setErrorTextField(false)
        if (viewModel.nextWord()) {
            updateNextWordOnScreen()
        } else {
            showFinalScoreDialog()
        }
    } else {
        setErrorTextField(true)
    }
}
```

6. Run your app. Play through some words. if the player's word is correct, the word is cleared on clicking the **Submit** button, otherwise a message saying "Try again!" is displayed. Notice that the **Skip** button is still not functional. You will add this implementation in the next task.

2:30



Unscramble

0 of 10 words

SCORE: 0

likls

Unscramble the word using all the letters.

Enter your word

skils



Try again!

SKIP

SUBMIT



10. Implement the Skip button

In this task, you add the implementation for `onSkipWord()` which handles when the **Skip** button is clicked.

1. Similar to `onSubmitWord()`, add a condition in the `onSkipWord()` method. If `true`, display the word on screen and reset the text field. If `false` and there's no more words left in this round, show the alert dialog with the final score.

```
/*
 * Skips the current word without changing the score.
 */
private fun onSkipWord() {
    if (viewModel.nextWord()) {
        setErrorTextField(false)
        updateNextWordOnScreen()
    } else {
        showFinalScoreDialog()
    }
}
```

2. Run your app. Play the game. Notice the **Skip** and **Submit** buttons are working as intended. Excellent!

11. Verify the ViewModel preserves data

For this task, add logging in `GameFragment` to observe that your app data is preserved in the `ViewModel`, during configuration changes. To access `currentWordCount` in `GameFragment`, you need to expose a read-only version using a backing property.

1. In `GameViewModel`, right click on the variable `currentWordCount`, select **Refactor** > **Rename...** . Prefix the new name with an underscore, `_currentWordCount`.
2. Add a backing field.

```
private var _currentWordCount = 0
val currentWordCount: Int
    get() = _currentWordCount
```

3. In `GameFragment` inside `onCreateView()`, above the return statement add another log to print the app data, word, score, and word count.

```
Log.d("GameFragment", "Word: ${viewModel.currentScrambledWord} " +
    "Score: ${viewModel.score} WordCount: ${viewModel.currentWordCount}")
```

4. In Android Studio open **Logcat**, filter on `GameFragment`. Run your app and play through some words. Change the orientation of your device. The fragment (UI controller) is destroyed and recreated. Observe the logs. Now you can see the score and word count increasing!

```

com.example.android.unscramble D/GameFragment: GameFragment created/re-
created!
com.example.android.unscramble D/GameFragment: GameViewModel created!
com.example.android.unscramble D/GameFragment: Word: oimfnru Score: 0
WordCount: 1
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-
created!
com.example.android.unscramble D/GameFragment: Word: ofx Score: 80 WordCount:
5
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-
created!
com.example.android.unscramble D/GameFragment: Word: ofx Score: 80 WordCount:
5
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-
created!
com.example.android.unscramble D/GameFragment: Word: nvoil Score: 160
WordCount: 9
com.example.android.unscramble D/GameFragment: GameFragment destroyed!
com.example.android.unscramble D/GameFragment: GameFragment created/re-
created!
com.example.android.unscramble D/GameFragment: Word: nvoil Score: 160
WordCount: 9

```

Notice that the app data is preserved in the `ViewModel` during orientation changes. You will update score value and word count in the UI using `LiveData` and Data Binding in later codelabs.

12. Update game restart logic

1. Run the app again, play the game through all the words. In the **Congratulations!** alert dialog, click **PLAY AGAIN**. The app won't let you play again because the word count has now reached the value `MAX_NO_OF_WORDS`. You need to reset the word count to 0 to play the game again from the beginning.
2. To reset the app data, in `GameViewModel` add a method called `reinitializeData()`. Set the score and word count to 0. Clear the word list and call `getNextWord()` method.

```

/*
 * Re-initializes the game data to restart the game.
 */
fun reinitializeData() {
    _score = 0
    _currentWordCount = 0
    wordsList.clear()
    getNextWord()
}

```

3. In `GameFragment` at the top the method `restartGame()`, make a call to the newly created method, `reinitializeData()`.

```
private fun restartGame() {  
    viewModel.reinitializeData()  
    setErrorTextField(false)  
    updateNextWordOnScreen()  
}
```

4. Run your app again. Play the game. When you reach the congratulations dialog, click on **Play Again**. Now you should be able to successfully play the game again!

This is what your final app should look like. The game shows ten random scrambled words for the player to unscramble. You can either **Skip** the word or guess a word and tap **Submit**. If you guess correctly, the score increases. An incorrect guess shows an error state in the text field. With each new word, the word count also increases.

Note that the score and word count displayed on screen do not update yet. But the information is still being stored in the view model and preserved during configuration changes like device rotation. You will update the score and word count on screen in later codelabs.

7:11



Unscramble

0 of 10 words

SCORE: 0

onmo

Unscramble the word using all the letters.

Enter your word

moon

SKIP

SUBMIT



7:11



Unscramble

0 of 10 words

SCORE: 0

mrtie

Unscramble the word using all the letters.

Enter your word

time



Try again!

SKIP

SUBMIT



At the end of 10 words, the game is over and an alert dialog pops up with your final score and an option to exit the game or play again.

7:11



Unscramble

leetapnh

Unscramble the word using all the letters.

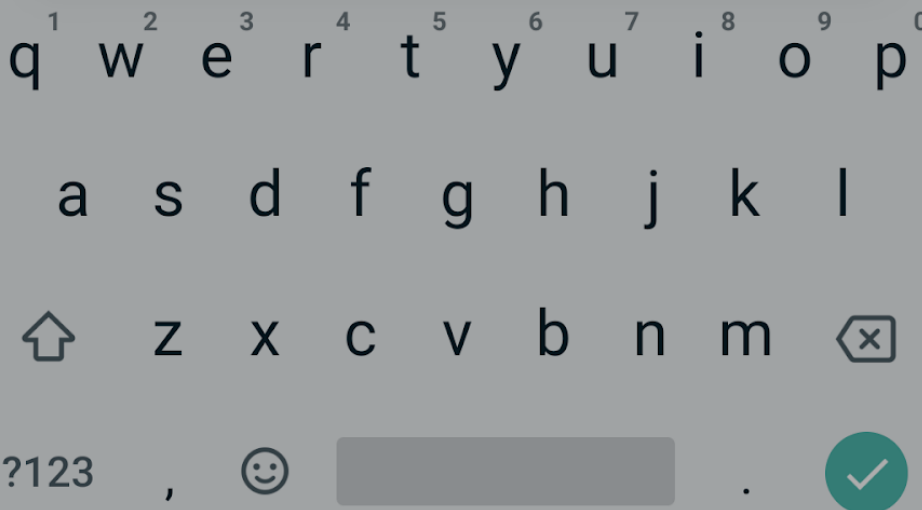
Enter your word

Congratulations!

You scored: 40

EXIT

PLAY AGAIN



Congratulations! You have created your first `ViewModel` and you saved the data!

13. Solution code

NOTE: Make sure to always include the package name of your app in all the Kotlin source files.

GameFragment.kt

```
import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import com.example.android.unscramble.R
import com.example.android.unscramble.databinding.GameFragmentBinding
import com.google.android.material.dialog.MaterialAlertDialogBuilder

/**
 * Fragment where the game is played, contains the game logic.
 */
class GameFragment : Fragment() {

    private val viewModel: GameViewModel by viewModels()

    // Binding object instance with access to the views in the
    game_fragment.xml layout
    private lateinit var binding: GameFragmentBinding

    // Create a ViewModel the first time the fragment is created.
    // If the fragment is re-created, it receives the same GameViewModel
    instance created by the
    // first fragment

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        // Inflate the layout XML file and return a binding object instance
        binding = GameFragmentBinding.inflate(inflater, container, false)
        Log.d("GameFragment", "GameFragment created/re-created!")
        Log.d("GameFragment", "Word: ${viewModel.currentScrambledWord} " +
            "Score: ${viewModel.score} WordCount:
${viewModel.currentWordCount}")
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        // Setup a click listener for the Submit and Skip buttons.
        binding.submit.setOnClickListener { onSubmitWord() }
        binding.skip.setOnClickListener { onSkipWord() }
```

```

        // Update the UI
        updateNextWordOnScreen()
        binding.score.text = getString(R.string.score, 0)
        binding.wordCount.text = getString(
            R.string.word_count, 0, MAX_NO_OF_WORDS)
    }

    /*
    * Checks the user's word, and updates the score accordingly.
    * Displays the next scrambled word.
    * After the last word, the user is shown a Dialog with the final score.
    */
    private fun onSubmitWord() {
        val playerWord = binding.textInputEditText.text.toString()

        if (viewModel.isUserWordCorrect(playerWord)) {
            setErrorTextField(false)
            if (viewModel.nextWord()) {
                updateNextWordOnScreen()
            } else {
                showFinalScoreDialog()
            }
        } else {
            setErrorTextField(true)
        }
    }

    /*
    * Skips the current word without changing the score.
    */
    private fun onSkipWord() {
        if (viewModel.nextWord()) {
            setErrorTextField(false)
            updateNextWordOnScreen()
        } else {
            showFinalScoreDialog()
        }
    }

    /*
    * Gets a random word for the list of words and shuffles the letters in
it.
    */
    private fun getNextScrambledWord(): String {
        val tempWord = allWordsList.random().toCharArray()
        tempWord.shuffle()
        return String(tempWord)
    }

    /*
    * Creates and shows an AlertDialog with the final score.
    */
    private fun showFinalScoreDialog() {
        MaterialAlertDialogBuilder(requireContext())
            .setTitle(getString(R.string.congratulations))
            .setMessage(getString(R.string.you_scored, viewModel.score))
            .setCancelable(false)
    }

```

```

        .setNegativeButton(getString(R.string.exit)) { _, _ ->
            exitGame()
        }
        .setPositiveButton(getString(R.string.play_again)) { _, _ ->
            restartGame()
        }
        .show()
    }

    /**
     * Re-initializes the data in the ViewModel and updates the views with
the new data, to
     * restart the game.
     */
    private fun restartGame() {
        viewModel.reinitializeData()
        setErrorTextField(false)
        updateNextWordOnScreen()
    }

    /**
     * Exits the game.
     */
    private fun exitGame() {
        activity?.finish()
    }

    override fun onDetach() {
        super.onDetach()
        Log.d("GameFragment", "GameFragment destroyed!")
    }

    /**
     * Sets and resets the text field error status.
     */
    private fun setErrorTextField(error: Boolean) {
        if (error) {
            binding.textField.isErrorEnabled = true
            binding.textField.error = getString(R.string.try_again)
        } else {
            binding.textField.isErrorEnabled = false
            binding.textInputEditText.text = null
        }
    }

    /**
     * Displays the next scrambled word on screen.
     */
    private fun updateNextWordOnScreen() {
        binding.textViewUnscrambledWord.text = viewModel.currentScrambledWord
    }
}

```

GameViewModel.kt

```

import android.util.Log
import androidx.lifecycle.ViewModel

/**
 * ViewModel containing the app data and methods to process the data
 */
class GameViewModel : ViewModel(){
    private var _score = 0
    val score: Int
        get() = _score

    private var _currentWordCount = 0
    val currentWordCount: Int
        get() = _currentWordCount

    private lateinit var _currentScrambledWord: String
    val currentScrambledWord: String
        get() = _currentScrambledWord

    // List of words used in the game
    private var wordsList: MutableList<String> = mutableListOf()
    private lateinit var currentWord: String

    init {
        Log.d("GameFragment", "GameViewModel created!")
        getNextWord()
    }

    override fun onCleared() {
        super.onCleared()
        Log.d("GameFragment", "GameViewModel destroyed!")
    }

    /**
     * Updates currentWord and currentScrambledWord with the next word.
     */
    private fun getNextWord() {
        currentWord = allWordsList.random()
        val tempWord = currentWord.toCharArray()
        tempWord.shuffle()

        while (String(tempWord).equals(currentWord, false)) {
            tempWord.shuffle()
        }
        if (wordsList.contains(currentWord)) {
            getNextWord()
        } else {
            _currentScrambledWord = String(tempWord)
            ++_currentWordCount
            wordsList.add(currentWord)
        }
    }

    /**
     * Re-initializes the game data to restart the game.
     */
    fun reinitializeData() {

```

```

        _score = 0
        _currentWordCount = 0
        wordsList.clear()
        getNextWord()
    }

    /**
     * Increases the game score if the player's word is correct.
     */
    private fun increaseScore() {
        _score += SCORE_INCREASE
    }

    /**
     * Returns true if the player word is correct.
     * Increases the score accordingly.
     */
    fun isUserWordCorrect(playerWord: String): Boolean {
        if (playerWord.equals(currentWord, true)) {
            increaseScore()
            return true
        }
        return false
    }

    /**
     * Returns true if the current word count is less than MAX_NO_OF_WORDS
     */
    fun nextWord(): Boolean {
        return if (_currentWordCount < MAX_NO_OF_WORDS) {
            getNextWord()
            true
        } else false
    }
}

```

14. Summary

- The Android app architecture guidelines recommend separating classes that have different responsibilities and driving the UI from a model.
- A UI controller is a UI-based class like `Activity` or `Fragment`. UI controllers should only contain logic that handles UI and operating system interactions; they shouldn't be the source of data to be displayed in the UI. Put that data and any related logic in a `ViewModel`.
- The `ViewModel` class stores and manages UI-related data. The `ViewModel` class allows data to survive configuration changes such as screen rotations.
- `ViewModel` is one of the recommended Android Architecture Components.

15. Learn more

- [ViewModel Overview](#)

- [Guide to app architecture](#)
- [Hands-on with Material Components for Android: Dialogs](#)
- [Alert dialog anatomy](#)
- [MaterialAlertDialogBuilder](#)
- [Backing Properties](#)
- [Android Architecture Components](#)
- [Android Material Dialogs](#)
- [Properties and Fields: Getters, Setters, const, lateinit](#)